NLP Preprocessing: Interview Preparation Master Guide

কমপ্লিট প্রশ্ন ও ইন্টারভিউ-রেডি বাংলা সমাধান (Question 1 - 10)

১. Corpus (কর্পাস)

What is a corpus?

কর্পাস (Corpus) হলো ভাষাগত বিশ্লেষণ, মডেল ট্রেনিং এবং ইভালুয়েশনের জন্য নির্দিষ্ট নিয়মে সংগ্রহ করা ডিজিটাল টেক্সট বা অডিও ডেটার বৃহৎ ভাণ্ডার।

Is the IMDB movie review dataset considered a corpus? Why?

হ্যাঁ, এটি একটি ক্লাসিক কর্পাস। কারণ এতে ৫০,০০০ টি সিনেমার রিভিউ (টেক্সট) নির্দিষ্ট কাঠামোর অধীনে সুসংগঠিতভাবে সংরক্ষিত রয়েছে, যা ন্যাচারাল ল্যাঙ্গুয়েজ প্রসেসিংয়ের সেন্টিমেন্ট অ্যানালাইসিস টাস্কে ব্যাপকভাবে ব্যবহৃত হয়।

২. Lowercasing (অক্ষর ছোট হাতের করা)

Why do we perform lowercasing?

সব ক্যাপিটাল ক্যারেক্টারকে স্মল লেটারে রূপান্তর করলে একই শব্দের একাধিক রূপ (যেমন: "PHITRON" এবং "phitron") সমান হিসেবে চিহ্নিত হয়, যা মডেলের ধারাবাহিকতা বজায় রাখে।

Output for: "PHITRON is Amazing! I love studying NLP at PHITRON."

```
phitron is amazing! i love studying nlp at phitron.
```

How does lowercasing help reduce vocabulary size?

যদি 'Phitron', 'PHITRON', এবং 'phitron'-কে আলাদা রাখা হয়, তবে ভোকাবুলারি সাইজ হবে ৩। লোয়ারকেসিং করার পর এরা ৩টি না হয়ে মাত্র ১টি টোকেন গঠন করে, ফলে ইউনিক টোকেন সংখ্যা ও কম্পিউটেশনাল ওভারহেড অনেক কমে যায়।

৩. HTML Tag Removal

Regex pattern for HTML tag removal: `<.*?>`

Why does `<.*?>` work?

এটি শুরু হয় `<` দিয়ে এবং শেষ হয় `>` দিয়ে। মাঝের যেকোনো ক্যারেক্টার `.` কে শূন্য বা ততধিকবার `*` ম্যাচ করায়।

Purpose of `?` in this regex:

এটি Non-greedy/Lazy matching নিশ্চিত করে। `?` না থাকলে পুরো বাক্যের প্রথম `<` থেকে শেষ `>` পর্যন্ত সব টেক্সটই একসাথে মুছে যেত (Greedy Match)। `?` দেওয়ার ফলে প্রতিটি ট্যাগ পৃথকভাবে ম্যাচ করে মুছে যায়।

৪. URL Removal

Regex Pattern: `r'https?://\S+|www\.\S+'`

- **https?:** এটি `http` অথবা `https` উভয় প্রোফোকলকেই ম্যাচ করে (`s` থাকা বা না থাকা ঐচ্ছিক)।
- **\S+:** এটি অন্তত একটি বা একাধিক Non-whitespace ক্যারেক্টারকে নির্দেশ করে (অর্থাৎ স্পেস পাওয়ার আগ পর্যন্ত পুরো ইউআরএল অ্যাড্রেসটি টেনে নেয়)।
- **www\.:** এটি সরাসরি `www.` দিয়ে শুরু হওয়া ওয়েব লিংক চেনার জন্য ব্যবহৃত হয়।

৫. Punctuation Removal

Role of `string.punctuation`:

এটি পাইথনের একটি বিল্ট-ইন কনস্ট্যান্ট যা সকল স্ট্যান্ডার্ড বিরামচিহ্ন যেমন: `!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~` বহন করে।

Why are the first two arguments of `maketrans()` empty strings?

`str.maketrans(x, y, z)` এ প্রথম দুটি আর্গুমেন্ট ব্যবহৃত হয় এক ক্যারেক্টারকে অন্য ক্যারেক্টারে ম্যাপিং বা পরিবর্তন করার জন্য। এখানে আমরা কোনো ক্যারেক্টার পরিবর্তন করতে চাই না, কেবল ৩য় আর্গুমেন্ট `z` এ থাকা চিহ্নগুলো সরাসরি মুছে ফেলতে (delete) চাই।

What if punctuation is not removed before tokenization?

বিরামচিহ্ন না মুছলে "Dhaka!" এবং "Dhaka" আলাদা ২টি ভিন্ন টোকেন হিসেবে কাউন্ট হবে, যা অযথা ভোকাবুলারি সাইজ বাড়িয়ে দেয় এবং নোয়েজ সৃষ্টি করে।

৬. Stopword Removal

What are stopwords?

যেসব শব্দ বাক্যের ব্যাকরণ তৈরি করতে অহরহ ব্যবহৃত হলেও বাক্যের মূল ভাব বা ইনফরমেশন বহনে খুব একটা ভূমিকা রাখে না (যেমন: is, the, in, am)।

Removed words from "I am studying Machine Learning in the university.":

স্টপওয়ার্ড হিসেবে বাদ যাবে: `I`, `am`, `in`, `the`।

Why convert stopwords into a set instead of keeping them as a list?

পাইথনে লিস্টে কোনো শব্দ আছে কি না খুঁজতে টাইম কমপ্লেক্সিটি হলো `O(N)`। অন্যদিকে সেটে (Set) হাশ টেবিল ব্যবহারের কারণে শব্দ খোঁজার টাইম কমপ্লেক্সিটি হলো `O(1)`। কোটি কোটি শব্দের কর্পাসে সেট ব্যবহার করলে পারফরম্যান্স বহুগুণ দ্রুত হয়।

৭. Tokenization

What is tokenization?

একটি অনুচ্ছেদ বা বাক্যকে ছোট ছোট প্রক্রিয়াজাতকরণযোগ্য একক (যাকে টোকেন বা শব্দ বলা হয়) এ বিভক্ত করার প্রক্রিয়া।

Difference between `text.split()` and `word_tokenize(text)`:

`text.split()` কেবল স্পেস দিয়ে শব্দ আলাদা করে। কিন্তু `word_tokenize()` ব্যাকরণিক নিয়ম মেনে বিরামচিহ্ন ও কনট্রাকশনকেও পৃথক টোকেন হিসেবে নিখুঁতভাবে হ্যান্ডেল করে।

Which handles punctuation better?

`word_tokenize()` বিরামচিহ্ন অনেক ভালোভাবে হ্যান্ডেল করতে পারে।

৮. Stemming

What is stemming?

অ্যালগরিদমীয় নিয়মের মাধ্যমে কোনো শব্দের প্রত্যয় (suffix) কেটে তার মূল মূলে (Stem) রূপান্তর করার দ্রুত প্রক্রিয়া।

Stemming output for words:

- running → `run` (বা `runn` ডিফেন্স অ্যালগরিদম ভেদে)
- studies → `studi`
- playing → `play`
- happiness → `happi`

Why produces words not in a dictionary?

স্টেমার অভিধানের অর্থ বা গ্রামার বোঝে না; এটি কেবল নির্দিষ্ট নিয়ম বা প্যাটার্নে কাটে (যেমন: -ies থাকলে -i করা)। তাই এটি অনেক সময় 'studi' বা 'happi'-এর মতো ব্যাকরণহীন শব্দ তৈরি করে।

৯. Lemmatization

What is lemmatization?

লেমাটাইজেশন হলো অভিধান (Vocabulary) এবং ব্যাকরণের নিয়ম ব্যবহার করে কোনো শব্দের পদের সঠিক মূল রূপ বা 'Lemma' খুঁজে বের করার প্রক্রিয়া।

Lemma for given words:

মূল শব্দ	স্টেমার (Porter)	লেমা (WordNet Lemmatizer)
running	runn	**run** (Verb হিসেবে)
children	children	**child** (Noun হিসেবে)
better	better	**good** (Adjective হিসেবে)
studies	studi	**study** (Noun/Verb হিসেবে)

Why is lemmatization generally more accurate than stemming?
স্টেমমিং ব্যাকরণ না বুঝে অন্ধভাবে অক্ষরের শেষাংশ কেটে ফেলে। কিন্তু লেমাটাইজেশন ডিকশনারি এবং POS (Part of Speech) ট্যাগ ব্যবহার করে শব্দের প্রকৃত ও অর্থপূর্ণ রূপ প্রদান করে (যেমন: 'better' কে 'good' বানানো স্টেমারের পক্ষে সম্ভব নয়)।

১০. Complete NLP Pipeline (ধাপভিত্তিক উদাহরণ)

Input Sentence:

```
OMG!!! I was studying NLP at PHITRON 😊. Visit https://phitron.io for more details.
```

ধাপভিত্তিক আউটপুট:

1. **Lowercasing:**
   `omg!!! i was studying nlp at phitron 😊. visit https://phitron.io for more details.`
2. **URL Removal:**
   `omg!!! i was studying nlp at phitron 😊. visit for more details.`
3. **Emoji Removal:**
   `omg!!! i was studying nlp at phitron . visit for more details.`
4. **Punctuation Removal:**
   `omg i was studying nlp at phitron visit for more details`
5. **Tokenization:**
   `['omg', 'i', 'was', 'studying', 'nlp', 'at', 'phitron', 'visit', 'for', 'more', 'details']`
6. **Stopword Removal:**
   `['omg', 'studying', 'nlp', 'phitron', 'visit', 'details']`
7. **Stemming (PorterStemmer):**
   `['omg', 'studi', 'nlp', 'phitron', 'visit', 'detail']`
8. **Lemmatization (WordNetLemmatizer):**
   `['omg', 'study', 'nlp', 'phitron', 'visit', 'detail']`